

Contents

Driver API Reference	1
How to generate a PDF	1
Shared hardware (both drivers)	1
Part 1 — LCD / monitor driver	2
void LCD_Init(I2C_HandleTypeDef *hi2c)	2
void LCD_Sleep(void)	2
void LCD_Wake(void)	3
void LCD_WriteLine(uint8_t row, const char *s)	3
void Update_LCD_Standard(uint16_t dist)	3
void Update_LCD_Debug(uint8_t pot, uint16_t dist)	4
Private helpers (LCD)	4
Part 2 — VL53L1X / sensor driver	4
Status codes (VL53L1X_Status_t)	4
VL53L1X_Status_t VL53L1X_Init(I2C_HandleTypeDef *hi2c)	5
VL53L1X_Status_t VL53L1X_StartMeasurement(I2C_HandleTypeDef *hi2c)	5
VL53L1X_Status_t VL53L1X_StopMeasurement(I2C_HandleTypeDef *hi2c)	5
uint8_t VL53L1X_IsDataReady(I2C_HandleTypeDef *hi2c)	5
VL53L1X_Status_t VL53L1X_GetDistance(I2C_HandleTypeDef *hi2c, uint16_t *pDistanceMm)	6
VL53L1X_Status_t VL53L1X_ClearInterrupt(I2C_HandleTypeDef *hi2c)	6
Private helpers (VL53)	6
Register map used by this driver	6
Typical call order (application)	7

Driver API Reference

Project: MCSD distance demonstrator

Author: Akos Eross, Daniel Lidy **MCU:** STM32L432KC Nucleo-32

Drivers: 16×2 HD44780 LCD (PCF8574 backpack) and VL53L1X time-of-flight sensor

This document describes the student-written driver functions: parameters, return values, behaviour, and the HAL / hardware calls underneath. It is meant to be exported to PDF (see the last section). In-code comments in `lcd.c` / `vl53l1x_driver.c` are the short form of the same information.

How to generate a PDF

From this directory, with Pandoc installed:

```
pandoc Driver_API_Reference.md -o Driver_API_Reference.pdf --toc --toc-depth=3 -V geometry:margin=2.5cm
```

Shared hardware (both drivers)

Both peripherals sit on **I2C1**.

Signal	Nucleo pin	CubeMX / HAL
SCL	PB6 (D5)	GPIO_MODE_AF_OD, GPIO_PULLUP, AF4 I2C1
SDA	PB7 (D4)	same
Bus	I2C1	Fast Mode, 7-bit addressing, analogue filter on

Internal pull-ups are a backup; the Click / LCD shield should also have externals. The application never uses DMA on this bus: LCD EN pulses are single-byte writes, and the ToF and LCD share the same peripheral.

`MX_I2C1_Init()` in `main.c` must run before either `LCD_Init()` or `VL53L1X_Init()`. Timing register `0x00B07CB4` is the CubeMX Fast-Mode value for this 32 MHz clock tree.

Part 1 — LCD / monitor driver

Files: `Core/Inc/lcd.h`, `Core/Src/lcd.c`

The display is a standard HD44780 16×2 in **4-bit mode**, reached through a PCF8574 I2C expander (Funduino / “LCD backpack” pinout).

Expander bit	Function
P0	RS (0 = command, 1 = data)
P1	RW (always 0 = write)
P2	EN (strobe)
P3	Backlight
P4–P7	Data nibble (D4–D7)

7-bit I2C addresses tried at init: **0x27**, then **0x3F**. HAL sees them as 8-bit write addresses `0x4E` and `0x7E`.

```
void LCD_Init(I2C_HandleTypeDef *hi2c)
```

In	<code>hi2c</code> — I2C1 handle from <code>MX_I2C1_Init()</code>
Out	none
Returns	void (no status; a missing backpack simply NACKs later writes)

Behaviour

1. Store `hi2c` in a static pointer used by every later call.
2. `HAL_I2C_IsDeviceReady(..., 0x27<<1, 3, 100)` then `0x3F<<1`. First ACK wins.
3. `HAL_Delay(50)` — HD44780 power-on wait.
4. Four nibble writes `0x30`, `0x30`, `0x30`, `0x20` with datasheet delays (5 ms / 1 ms) to leave 8-bit reset and enter 4-bit mode.
5. Commands: `0x28` (2 lines, 5×8), `0x08` (display off), `0x01` (clear), `0x06` (entry mode), `0x0C` (display on, cursor off).

Hardware / HAL

- `HAL_I2C_IsDeviceReady`
- `HAL_I2C_Master_Transmit` (one byte per expander write, 100 ms timeout)
- `HAL_Delay` only here and for clear/home (`0x01` / `0x02`)

Must be configured: I2C1 as above; call once after `MX_I2C1_Init()`.

```
void LCD_Sleep(void)
```

In / out	none
Returns	void

Behaviour

1. Command 0x08 — display off (DDRAM kept).
2. Clear the static backlight bit.
3. Write expander 0x00 so the backlight MOSFET is off.

Used when the application idles 30 s (ST_SLEEP).

```
void LCD_Wake(void)
```

In / out	none
Returns	void

Behaviour

1. Restore the backlight bit.
2. Command 0x0C — display on, cursor off.

Used on SW1 wake from ST_SLEEP. The application then redraws the text.

```
void LCD_WriteLine(uint8_t row, const char *s)
```

In	row — 0 = top, 1 = bottom. s — C string, or NULL (blank line).
Out	none
Returns	void

Behaviour

1. Set DDRAM address: 0x80 + col (row 0) or 0xC0 + col (row 1), col = 0.
2. Write exactly **16** characters. Bytes from s until NUL, then spaces.

This always overwrites a whole row so leftover glyphs cannot remain.

Hardware: six I2C bytes per character (high nibble + low nibble, three expander writes each: data, EN high, EN low). EN high is held >450 ns with a busy-wait; after each nibble the driver waits ~50 µs (>37 µs HD44780 spec).

```
void Update_LCD_Standard(uint16_t dist)
```

In	dist — distance in millimetres
Out	none
Returns	void

Behaviour

- `cm = dist / 10`, clamped to 0...999.
- Formats "Dist: %3d cm" and calls `LCD_WriteLine(0, ...)`.
- Line 1 is not touched (the application writes the on-time countdown there).

```
void Update_LCD_Debug(uint8_t pot, uint16_t dist)
```

In	<code>pot</code> — 0...100 %. <code>dist</code> — millimetres.
Out	none
Returns	void

Behaviour

- Line 0: "DBG P:%3u%%".
- Line 1: "D:%4u mm" (mm clamped at 9999).

Kept as the published Dani API. The current application uses UART text entry in the 67–100 % pot band instead of this layout.

Private helpers (LCD)

Helper	Role
<code>lcd_delay_us</code>	Busy-wait from <code>SystemCoreClock</code> (EN pulse / nibble settle).
<code>lcd_i2c</code>	<code>HAL_I2C_Master_Transmit</code> of one expander byte.
<code>lcd_nibble</code>	Put D7–D4 + BL + RS, pulse EN.
<code>lcd_byte</code>	High nibble then low nibble.
<code>lcd_cmd / lcd_data</code>	RS = 0 / 1. Clear/home add <code>HAL_Delay(2)</code> .
<code>lcd_gotoxy</code>	DDRAM set (0x80 / 0xC0).

Part 2 — VL53L1X / sensor driver

Files: `Core/Inc/vl53l1x_driver.h`, `Core/Src/vl53l1x_driver.c`

This is a small register driver (not the ST ULD). The 8-bit HAL address is **0x52** (7-bit **0x29**). Registers are 16-bit addresses.

GPIO1/INT on the Click is **not** used as an EXTI line (it would preempt LCD nibble writes). Data-ready is polled in `VL53L1X_REG_INT_STATUS`.

Status codes (`VL53L1X_Status_t`)

Value	Name	Meaning
0	<code>VL53L1X_OK</code>	I2C succeeded and the sample is usable

	Value	Name	Meaning
	-1	VL53L1X_ERROR	HAL_I2C_Mem_* failed (NACK, bus, timeout)
	-2	VL53L1X_TIMEOUT	Boot bit in SYS_STATUS stayed 0 for >100 ms
	-3	VL53L1X_INVALID	Chip ID mismatch, or range status not accepted

VL53L1X_Status_t VL53L1X_Init(I2C_HandleTypeDef *hi2c)

In	hi2c — I2C1 handle
Out	none
Returns	OK / ERROR / TIMEOUT / INVALID

Behaviour

1. Read MODEL_ID (0x010F), MODULE_TYPE (0x0110), MASK_REVISION (0x0111). Expect 0xEA, 0xCC, 0x10.
2. Poll SYS_STATUS (0x0022) until non-zero, or 100 ms (HAL_GetTick).
3. Burst-write 91 bytes of ST default configuration at 0x002D.
4. HAL_Delay(5).

Hardware / HAL: HAL_I2C_Mem_Read / HAL_I2C_Mem_Write with I2C_MEMADD_SIZE_16BIT, 100 ms timeout.

VL53L1X_Status_t VL53L1X_StartMeasurement(I2C_HandleTypeDef *hi2c)

In	hi2c
Returns	OK or ERROR

Writes 0x01 to INT_CLEAR (0x0086), then 0x40 to SYS_START (0x0087) for continuous ranging.

VL53L1X_Status_t VL53L1X_StopMeasurement(I2C_HandleTypeDef *hi2c)

In	hi2c
Returns	OK or ERROR

Writes 0x00 to SYS_START. Called before ST_SLEEP so the sensor is quiet while the CPU waits on SW1 EXTI.

uint8_t VL53L1X_IsDataReady(I2C_HandleTypeDef *hi2c)

In	hi2c
Returns	1 if INT_STATUS bit 0 is set, else 0 (including I2C errors)

Read-only poll of register 0x0031. The application calls this every live loop iteration; if 0 it skips `GetDistance`.

```
VL53L1X_Status_t VL53L1X_GetDistance(I2C_HandleTypeDef *hi2c, uint16_t *pDistanceMm)
```

In	hi2c
Out	*pDistanceMm — millimetres (big-endian register 0x0096), written whenever both I2C reads succeed
Returns	OK, ERROR, or INVALID

Behaviour

1. Read RANGE_STATUS (0x0089), keep the low 5 bits.
2. Read 16-bit RANGE_MM (0x0096).
3. Decode status with the ST STATUS_RTN_TABLE.
4. Return OK if decoded status is **0, 1, 2 or 7**:

Decoded	Meaning	Treated as
0	Range valid	OK
1	Sigma warning	OK (still a number)
2	Signal warning	OK
7	Wrap-around warning	OK
other	Phase, min-clip, ...	INVALID (sample discarded by the app)

The application converts millimetres to centimetres (mm / 10) and clamps to CM_MIN...CM_MAX (**1...500 cm**). Hold freezes that centimetre value; it does not re-read the sensor.

```
VL53L1X_Status_t VL53L1X_ClearInterrupt(I2C_HandleTypeDef *hi2c)
```

In	hi2c
Returns	OK or ERROR

Writes 0x01 to INT_CLEAR so the next sample can raise data-ready again. The live path always calls this after a ready poll, even if GetDistance returned INVALID.

Private helpers (VL53)

Helper	HAL	Role
I2C_WriteReg8	HAL_I2C_Mem_Write 1 byte	start / stop / clear
I2C_ReadReg8	HAL_I2C_Mem_Read 1 byte	ID, status, INT
I2C_ReadReg16	HAL_I2C_Mem_Read 2 bytes	distance, MSB first

Register map used by this driver

Address	Name	Use
0x0022	SYS_STATUS	boot complete
0x002D	(block)	91-byte default config
0x0031	INT_STATUS	data ready (bit 0)
0x0086	INT_CLEAR	write 0x01
0x0087	SYS_START	0x40 start, 0x00 stop
0x0089	RANGE_STATUS	5-bit raw status
0x0096	RANGE_MM	distance, millimetres
0x010F-0x0111	ID	0xEA / 0xCC / 0x10

Typical call order (application)

1. `MX_I2C1_Init()`
2. `VL53L1X_Init` → `VL53L1X_StartMeasurement` (via `tof_try_attach`)
3. `LCD_Init`
4. Live: `VL53L1X_IsDataReady` → `VL53L1X_GetDistance` → `VL53L1X_ClearInterrupt`
5. Sleep: `VL53L1X_StopMeasurement` then `LCD_Sleep`
6. Wake: `LCD_Wake`, then start measurement again when Live / UART TEST needs it